Instruction For Reproducing Section 4.3 Under PxG

Covariate-Assisted Bayesian Graph Learning for Heterogeneous Data

Description: This R Markdown file provides the workflow for reproducing simulation results in §4.3 under the PxG model.

1. Generating Simulation Data

```
rm(list = ls())

## loading libraries
library(BDgraph)
library(invgamma)

## generating simulation data from the 50-dimensional GGM in section 4.3
set.seed(2022)
p = 10
q = 50
n = 500
X = matrix(0, n, p)
for(i in 1:(n/2)){
  X[i,] = rnorm(p, 0, 1)
}
for(i in (n/2+1):n){
  X[i,] = rnorm(p, 2, 1)
}

Y = matrix(0, n, q)
Omega1 = matrix(0, q, q)
Omega1[upper.tri(Omega1, diag = FALSE)] = rbinom(q*(q-1)/2, 1, 0.0085)
Omega1 = Omega1 + t(Omega1); diag(Omega1) = 1; G1 = Omega1
Omega2 = matrix(0, q, q)
Omega2[upper.tri(Omega2, diag = FALSE)] = rbinom(q*(q-1)/2, 1, 0.0085)
Omega2 = Omega2 + t(Omega2); diag(Omega2) = 1; G2 = Omega2
Omega1 = rgwish(1,Omega1,3,diag(q))
Omega2 = rgwish(1,Omega2,3,diag(q))
Sigma1 = solve(Omega1)
Sigma2 = solve(Omega2)
for(i in 1:(n/2)){ Y[i,] = t(t(chol(Sigma1))%*%rnorm(q,0,1)) }
for(i in (n/2+1):n){ Y[i,] = t(t(chol(Sigma2))%*%rnorm(q,0,1)) }
```

2. Fitting Three Models: PxG, Covariate-only, Graph-only. Comparing DIC Values.

```
## loading libraries and compiling all C++ functions
library(Rcpp)
library(RcppArmadillo)
```

```
sourceCpp("PxG-PL.cpp")

## fitting simulation data with the proposed PxG model
set.seed(2022)
A = PxG_PL(YData = Y, XData = X,
           spike = 0.01, slab = 5, a1 = 1, a2 = 1, alphaG = 0.1,
           mu0 = rep(0, p), sigsq0 = 1, b1 = 1, b2 = 0.1,
           M = 1, N = 3, burnin = 500, nmcmc = 1000,
           z0 = sample(1:3,n,TRUE, c(1/3,1/3,1/3)), B = 1)

## fitting simulation data with the covariate-only model
set.seed(2022)
B = CovOnly_PL(YData = Y, XData = X,
               spike = 0.01, slab = 5, a1 = 1, a2 = 1, alphaG = 0.1,
               mu0 = rep(0, p), sigsq0 = 1, b1 = 1, b2 = 0.1,
               M = 1, N = 3, burnin = 500, nmcmc = 1000,
               z0 = sample(1:3,n,TRUE, c(1/3,1/3,1/3)))

## fitting simulation data with the graph-only model
set.seed(2022)
C = GraphOnly_PL(YData = Y, XData = X,
                 spike = 0.01, slab = 5, a1 = 1, a2 = 1, alphaG = 0.1,
                 mu0 = rep(0, p), sigsq0 = 1, b1 = 1, b2 = 0.1,
                 M = 1, N = 3, burnin = 500, nmcmc = 1000,
                 z0 = sample(1:3,n,TRUE, c(1/3,1/3,1/3)), B = 1)
```

```
# DIC for the PxG model
A$DIC
```

```
## [1] 104164.7
```

```
# DIC for the covariate-only model
B$DIC
```

```
## [1] 155453.3
```

```
# DIC for the graph-only model
C$DIC
```

```
## [1] 106138.6
```

3. Generating Figure 7

```
## loading libraries for plot
library(ggplot2)
library(ggpubr)

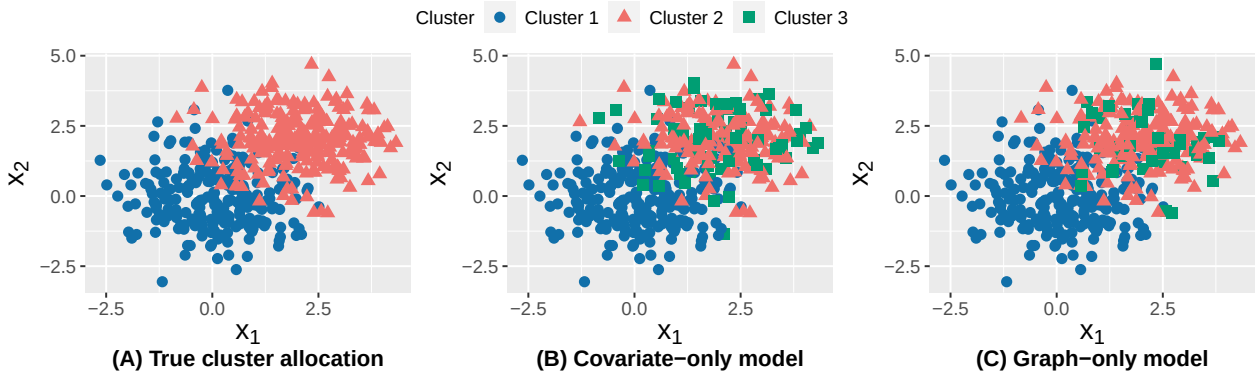
## plot Figure 7
subX = as.data.frame(X[,1:2])
subX[501,] = c(NA, NA)
subX$Cluster = c(rep("Cluster 1",n/2), rep("Cluster 2",n/2), rep("Cluster 3", 1))
cols <- c("#1170AA", "#EF6F6A", "#009E73")
fg1 = ggplot(subX, aes(x = V1, y = V2, color = Cluster,shape = Cluster) ) +
  geom_point(size = 2.5) + scale_color_manual(values = cols) +
  theme(axis.text = element_text(size = 10), axis.title = element_text(size = 15),
        legend.text = element_text(size = 10), legend.title = element_text(size = 10),
```

```
        plot.caption = element_text(color="black", size=12,
                                    face="bold", hjust = 0.5, vjust = 3)) +
  labs(x = expression(x[1]), y = expression(x[2]), caption="(A) True cluster allocation")

subX = as.data.frame(X[,1:2])
Bind = rep(0,500)
Bind[which(B$Label==2)] = "Cluster 1"
Bind[which(B$Label==1)] = "Cluster 2"
Bind[which(B$Label==3)] = "Cluster 3"
subX$Cluster = Bind
cols <- c("#1170AA", "#EF6F6A", "#009E73")
fg2 = ggplot(subX, aes(x = V1, y = V2, color = Cluster, shape = Cluster) ) +
  geom_point(size = 2.5) + scale_color_manual(values = cols) +
  theme(axis.text = element_text(size = 10), axis.title = element_text(size = 15),
        legend.text = element_text(size = 10), legend.title = element_text(size = 10),
        plot.caption = element_text(color="black", size=12,
                                    face="bold", hjust = 0.5, vjust = 3)) +
  labs(x = expression(x[1]), y = expression(x[2]), caption="(B) Covariate-only model")

subX = as.data.frame(X[,1:2])
Cind = rep(0,500)
Cind[which(C$Label==2)] = "Cluster 1"
Cind[which(C$Label==1)] = "Cluster 2"
Cind[which(C$Label==3)] = "Cluster 3"
subX$Cluster = Cind
cols <- c("#1170AA", "#EF6F6A", "#009E73")
fg3 = ggplot(subX, aes(x = V1, y = V2, color = Cluster, shape = Cluster) ) +
  geom_point(size = 2.5) + scale_color_manual(values = cols) +
  theme(axis.text = element_text(size = 10), axis.title = element_text(size = 15),
        legend.text = element_text(size = 10), legend.title = element_text(size = 10),
        plot.caption = element_text(color="black", size=12,
                                    face="bold", hjust = 0.5, vjust = 3)) +
  labs(x = expression(x[1]), y = expression(x[2]), caption="(C) Graph-only model")

fg0 = ggarrange(fg1, fg2, fg3, ncol = 3, nrow = 1, common.legend = TRUE); fg0
```



(A) True cluster allocation (B) Covariate−only model (C) Graph−only model

4. Calculating graph estimate for each cluster under the PxG model

```
set.seed(2022)
AG1 = Graph_NS(YData = Y[which(A$Label==2),], spike = 0.01, slab = 5,
               a1 = 1, a2 = 1, alphaG = 0.05, burnin = 100, nmcmc = 500)

set.seed(2022)
AG2 = Graph_NS(YData = Y[which(A$Label==3),], spike = 0.01, slab = 5,
               a1 = 1, a2 = 1, alphaG = 0.05, burnin = 100, nmcmc = 500)
```

```
## making pseudo-likelihood graph estimates symmetric
AG1new = AG1
for(i in 1:50){
  for(j in 1:50){
    AG1new[i,j] = max(AG1[i,j], AG1[j,i])
  }
}
diag(AG1new) = 1

AG2new = AG2
for(i in 1:50){
  for(j in 1:50){
    AG2new[i,j] = max(AG2[i,j], AG2[j,i])
  }
}
diag(AG2new) = 1

xaxis = as.character(1:50)
yaxis = xaxis[50:1]
data = expand.grid(X=xaxis, Y=yaxis)
diag(G1) = 0
diag(G2) = 0
```

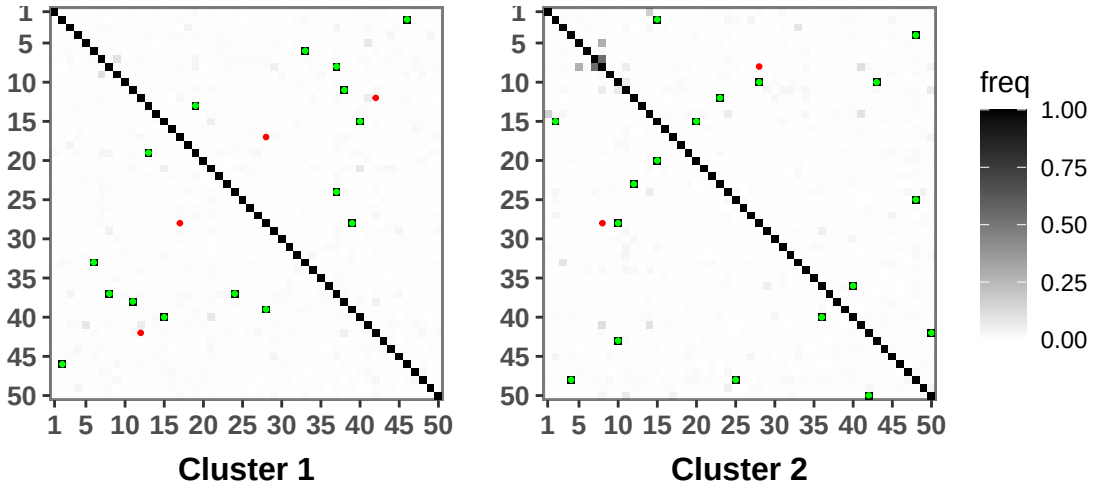
5. Generating Figure 8

```
G1T = G1F = matrix(0,50,50)
for(i in 1:50){
  for(j in 1:50){
    if( (G1[i,j]==1)&(AG1new[i,j]>=0.5) ){ G1T[i,j] = 1 }
    if( (G1[i,j]==1)&(AG1new[i,j]<0.5) ){ G1F[i,j] = 1 }
  }
}
gg1 = AG1new[50:1,]
data$freq = as.vector(t(gg1))
data$vT = as.vector(t(G1T[50:1,]))
data$vF = as.vector(t(G1F[50:1,]))
fg1 = ggplot(data, aes(X, Y, fill = freq)) + geom_tile() + labs(y="", x = "Cluster 1") +
      theme(axis.text.x = element_text(angle = 0, vjust = 0.75, hjust = 0.5)) +
      theme(axis.text.y = element_text(angle = 0, hjust = 0.75, vjust = 0.5)) +
      scale_fill_continuous(high = "black", low = "white") +
      scale_x_discrete("Cluster 1",
                       c("1", "5", "10", "15", "20", "25", "30", "35", "40", "45", "50")) +
      scale_y_discrete("",
```

```
                        c("1", "5", "10", "15", "20", "25", "30", "35", "40", "45", "50")) +
      theme(axis.text=element_text(size=10, face = "bold"),
            axis.title=element_text(size=12, face="bold"),
            legend.position="none",
            axis.title.x = element_text(margin = margin(c(7,0,0,0))),
            aspect.ratio=1, plot.margin=unit(c(0,-3,0,-3.65), "cm")) +
      geom_point(col = "green", aes(size=ifelse(vT, "dot", "no_dot"))) +
      geom_point(col = "red", aes(size=ifelse(vF, "dot", "no_dot"))) +
      scale_size_manual(values=c(dot=0.5, no_dot=NA), guide="none") +
      theme(panel.border=element_rect(fill = NA, colour=alpha('black', .5),size=0.75))

G2T = G2F = matrix(0,50,50)
for(i in 1:50){
  for(j in 1:50){
    if( (G2[i,j]==1)&(AG2new[i,j]>=0.5) ){ G2T[i,j] = 1 }
    if( (G2[i,j]==1)&(AG2new[i,j]<=0.5) ){ G2F[i,j] = 1 }
  }
}
gg2 = AG2new[50:1,]
data$freq = as.vector(t(gg2))
data$vT = as.vector(t(G2T[50:1,]))
data$vF = as.vector(t(G2F[50:1,]))
fg2 = ggplot(data, aes(X, Y, fill = freq)) + geom_tile() + labs(y="", x = "Cluster 2") +
  theme(axis.text.x = element_text(angle = 0, vjust = 0.75, hjust = 0.5)) +
  theme(axis.text.y = element_text(angle = 0, hjust = 0.75, vjust = 0.5)) +
  scale_fill_continuous(high = "black", low = "white") +
  scale_x_discrete("Cluster 2",
                   c("1", "5", "10", "15", "20", "25", "30", "35", "40", "45", "50")) +
  scale_y_discrete("",
                   c("1", "5", "10", "15", "20", "25", "30", "35", "40", "45", "50")) +
  theme(axis.text=element_text(size=10, face = "bold"),
        axis.title=element_text(size=12, face="bold"),
        legend.position="right",
        axis.title.x = element_text(margin = margin(c(7,0,0,0))),
        aspect.ratio=1, plot.margin=unit(c(0,-3,0,-3.65), "cm")) +
  geom_point(col = "green", aes(size=ifelse(vT, "dot", "no_dot"))) +
  geom_point(col = "red", aes(size=ifelse(vF, "dot", "no_dot"))) +
  scale_size_manual(values=c(dot=0.5, no_dot=NA), guide="none") +
  theme(panel.border=element_rect(fill = NA, colour=alpha('black', .5),size=0.75))

fg0 = ggarrange(fg1, fg2, ncol = 2, nrow = 1, common.legend = FALSE); fg0
```


1
5
10
15
20
25
30
35
40
45
50
1 5 10 15 20 25 30 35 40 45 50
Cluster 1
Cluster 2
freq
1.00
0.75
0.50
0.25
0.00